ФЕДЕРАЛЬНОЕ АГЕНТСТВО ЖЕЛЕЗНОДОРОЖНОГО ТРАНСПОРТА
федеральное государственное бюджетное образовательное
учреждение высшего образования
«ОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПУТЕЙ СООБЩЕНИЯ»
(ОмГУПС (ОмИИТ))

Кафедра «Автоматика и системы управления»

ПРОГРАММИРОВАНИЕ ИГРЫ НА ЯЗЫКЕ С

Пояснительная записка к курсовой работе
по дисциплине «Программирование»

ИНМВ.400006.000 ПЗ

Студент гр. 22м
_____ Кучеренко Е.А.
30.05.2023

Руководитель –
доцент кафедры АиСУ
_____ Пономарев А. В.
«__»_____2023 г.

Омск 2023

Реферат

УДК 004.42

Пояснительная записка к курсовой работе содержит 30 страниц, 27 рисунков, 3 использованных источника, 2 приложения.

Объектом курсовой работы является консольная игра «Наборщик».

Цель курсовой работы – получение основных навыков использования языка Си, создание игры с искусственным интеллектом.

Результатом курсовой работы является игра «Наборщик», написанная на языке Си в программе Visual Studio Code

В процессе создания игры была изучена лексика языка Си.

Пояснительная записка выполнена в текстовом редакторе Microsoft Word 2003.

Содержание

Введение 4
1 Правила игры 5
2 Реализация игры 6
3 Инструкция пользователя 10
Заключение 13
Библиографический список 14
Приложение А 15
Приложение Б 27

Введение

Язык программирования С – универсальный язык программирования, который завоевал особую популярность у программистов, благодаря сочетанию возможностей языков программирования высокого и низкого уровней. Основная причина, по которой большинство разработчиков предпочитают использовать Си для серьезных проектов – свобода выражения мыслей, мобильность и доступность языка.

Деннис Ритчи разработал язык программирования Си в 1970–х годах в лаборатории Bell Labs с целью заменить сложный для использования Ассемблер. В 1983 году был выпущен стандарт Си ANSI C, который существенно упростил и унифицировал структуру языка. Этот индустриальный язык программирования может использоваться на любой платформе и легко осваивается благодаря своему небольшому набору операций и конструкций. Часть отсутствующих возможностей относительно легко имитируется встроенными средствами, часть добавляется с помощью сторонних библиотек, часть реализуется в некоторых компиляторах в виде расширений языка.

Си широко используется для системного программирования, так как он позволяет управлять контроллерами и аппаратным обеспечением.

Несмотря на отсутствие автоматического управления памятью, Си является популярным языком, используемым в научных и инженерных приложениях, создании встроенных систем, игр и многих других областях.

Курсовая работа является примером использования языка Си.

1 Правила игры

Наборщик – одна из самых популярных интеллектуальных словесных игр, суть которой заключается в том, чтобы строить слова из набора случайных букв и набрать как можно больше очков. Игра «Наборщик» развивает внимание, грамотность и взаимно обогащает словарный запас игроков.

Игра с поиском слов может быть расслабляющим занятием для ума, но это не означает, что мозг переходит в режим отдыха, решая эти головоломки. Напротив – зона поиска слов заставляют мозг выполнять систематический поиск в любое время, будь то поиск определенного образца/окончания слова или конкретной буквы. Умение сосредоточиться и полностью сконцентрироваться на чем–то – это навык, который необходимо тренировать на регулярной основе.

Однажды, группа друзей решила провести вечер за настольной игрой. Они рассмотрели множество вариантов, ни один из которых им не понравился, и один из их друзей, работающий в IT–компании, предложил им создать свою собственную игру, основанную на процессе поиска слов.

Так начался процесс создания игры. Сначала были определены правила, и игроки должны были за определенное время набрать на клавиатуре слово на заданную тему. За каждое угаданное слово начислялись очки. Были предложены различные темы, включая фрукты, животных и географические объекты.

После многочисленных испытаний была создана идеальная игра для набора и поиска слов. Игра быстро стала популярной и получила много положительных отзывов. С тех пор игру интерпретировали по–разному. Одна из версий игры была разработана в ходе курсовой работы. Правила игры:

Участие принимают два игрока. В начале игры произвольно подбирается слово. Задача игроков, пользуясь буквами этого слова, составить новые слова. Соблюдая следующие правила:

1) Писать можно только имена существительные в именительном падеже, единственном числе, исключая имена собственные;

2) Каждое слово должно быть составлено только из тех букв, которые есть в слове.

3) Нельзя использовать слова, которые уже были воздействованы ранее. Игра заканчивается в тот момент, когда игроки больше не смогут сделать ход.

2 Реализация игры

В программе используются библиотеки:

#include <windows.h>//библиотека для работы с системой

#include <stdlib.h>//содержит определения и прототипы функций общего пользования

#include <locale.h>//отвечает за работу с локальной культурой

#include <string.h>//для работы со строками

#include <ctype.h>//отвечает за преобразование и проверку символов

#include <stdio.h>//для работы со средствами ввода–вывода

#include <wchar.h>//для работы с символами

#include <time.h>//для работы с датами и временем

Программа разработана с использованием структурного подхода к программированию. Для решения поставленной задачи были реализованы несколько функций.

Функции данной игры:

– Greeting() – функция для приветствия пользователя в игре;

– Rules() – функция для вывода правил игры;

– Farewell() – функция для завершения игры;

– Compare() – функция для сортировки турнирной таблицы;

– Parse() – функция разделения данных из текстового файла;

– FirstWritingToAFile() – функция для записи данных первого игрока в текстовый файл;

– SecondWritingToAFile() – функция для записи данных второго игрока в текстовый файл;

– Check() – функция, проверяющая существование слова;

– FirstCheck() – функция, проверяющая удовлетворяет ли слово требованию;

– SecondCheck() – функция, проверяющая повторялось ли слово;

– ThePlayersMove() – функция, осуществляющая ход пользователя;

– FillDictionary() – функция заполнения словаря;

– Score() – функция для вывода результатов игры;

– Rating() – функция для вывода турнирной таблицы.

Директива #define позволяет вводить в текст программы константы и макроопределения. Общая форма записи:

#define Идентификатор Замена

Директива #define указывает компилятору, что нужно подставить строку, определенную аргументом. Замена, вместо каждого аргумента Идентификатор в исходном файле.

Константы и макроопределения данной игры:

```
#define MAX_NUM_WORDS 68000
#define MAX_NUM_LETTERS 11
#define SIZE 20
#define DBOLD    "\033[2m"//серый цвет
#define GREEN    "\033[0;32m"//зеленый цвет
#define RESET    "\033[0m"//отмена настроек
#define RED      "\033[1;31m"//красный цвет
```

Для приветствия используется функция Greeting(). Также она спрашивает у пользователей, хотят ли они ознакомиться с правилами игры, начать игру или выйти. Если игроки выбирают ознакомиться с правилами, то функция вызывает функцию Rules(). Если пользователи принимают решение выйти из игры, то вызывается функция Farewell().

Функции Rules() и Farewell() ничего не принимают и ничего не возвращают.

Функции FirstWritingToAFile() и SecondWritingToAFile() работают по одному принципу. Внутри функций происходит запись данных первого и второго игроков в текстовый файл. В функции передается struct Date users.

Функция ThePlayersMove() осуществляет ход пользователя. Если ход некорректен, то выводится ошибка, и игроку предлагается ввести ход заново. Функция strcmp() побайтно сравнивает коды символов двух строк, на которые указывают аргументы функции. Сравнение продолжается до встречи первого отличающегося символа или пока не будут проверены все символы строк. strcmp() позволяет программе понять, хочет ли игрок закончить игру или же он нуждается в подсказке. На рисунке 1 напечатан код, который генерирует слово–подсказку. Слово выводится по индексу, который генерирует функция srand(time(NULL)). Генератор «случайных» на самом деле не случаен, а обычно задается формулой. Но формула при одних и тех–же значениях даёт естественно одинаковый результат. Начальное значение у этой формулы принято называть seed. Так вот srand инициализирует этот seed определенным значением. Но нам нужно, чтобы был разный seed. За это отвечает time(NULL), означает взять текущее время.

```
if (!strcmp(word, "hint")){
    if (*score < 15){
        printf("%sУ вас недостаточно очков! %s", RED, RESET);
        return 0;
    }
    else {
        while (n == 0){
            srand(time(NULL)); // инициализация генератора псевдослучайных чисел
            // int randomIndex = 0 + rand()%(MAX_NUM_WORDS - 1 + 1); // генерация случайного индекс
            memcpy(word, DICTIONARY_WORDS[rand() % MAX_NUM_WORDS - b], 11);
            resultFirstCheck = FirstCheck(playWord, word);
            if (resultFirstCheck == 1){
                resultSecondCheck = SecondCheck(ENTERED_WORDS, counter, word);
                if (resultSecondCheck == 0){
                    n++;
                }
            }
            b++;
        }
        printf("%s", DBOLD);
        printf("ПОДСКАЗКА   :       %s\n", word);
        printf("%s", RESET);
        *score -= 3;
        return 1;
    }
}
```

Рисунок 1 – Генератор слова–подсказки

Функция ThePlayersMove() вызывает функции FirstCheck() и SecondCheck().

Функция FirstCheck() проверяет все ли буквы из введённого слова находятся в игровом слове. Функция FirstCheck() вызывает функцию Check().

Функция Check() проверяет существование слова.

Функция SecondCheck() проверяет слова на повтор. Внутри функции SecondCheck() используется функция stricmp, она побайтно сравнивает коды символов двух строк, игнорирует разницу между символами в нижнем и верхнем регистре.

Функция FillDictionary() используется для заполнения массива словами из текстового файла. В функцию передается указатель на целочисленную переменную типа int.

Функция Score() – это функция для вывода результатов игры. В нее передаются переменные типа int, в которых хранятся результаты игры и struct Date users. Результаты первого игрока и второго сравниваются и, в результате, объявляется победитель.

Функция Rating() используется для вывода турнирной таблицы, внутри нее вызывается функция Parse().

Функция Parse(), осуществляет разделение данных из текстового файла. Внутри функции открывается текстовый файл data.txt, затем, массивы текстового файла делятся на строки name и score. С помощью функции atoi(), происходит преобразование строки score в число типа int. После преобразований, текстовый файл закрывается. Функция qsort выполняет сортировку n элементов массива, на который ссылается указатель f. Для каждого элемента массива устанавливается размер в байтах, который передается через параметр size. Последний параметр функции qsort — указатель Compare().

Функция Compare() – это функция сравнения, которая используется для определения порядка следования элементов в отсортированном массиве.

В главной функции main() сначала очищается экран консоли с помощью команды system(«cls»). Функция main() вызывает функцию Greeting(). Внутри функции существует переменная choice, которая отвечает за навигацию в игре. На рисунке 2 выведен код, позволяющий сгенерировать случайным способом игровое слово и поместить его в массив playWord, с помощью функции memcpy(). Функция memcpy() копирует определенное количество байт из одного блока памяти в другой.

```
FILE *file = fopen("dictionary.txt", "r");
char *fileContents[MAX_NUM_WORDS+1];
char buffer[MAX_NUM_WORDS + 1];
for (size_t i = 0; fgets(buffer, sizeof buffer, file); i++){
    if (strlen(buffer) == 11){
        n++;
        fileContents[n] = strdup(buffer);
    }
}
fclose(file);
srand(time(NULL)); // инициализация генератора псевдослучайных чисел
int randomIndex = 0 + rand()%(n - 1 + 1); // генерация случайного индекса
if (randomIndex == 0){
    randomIndex++;
}
memcpy(playWord, fileContents[randomIndex], 11);
result = FillDictionary(&countWords);
```

Рисунок 2 – Определение игрового слова

3 Инструкция пользователя

При запуске игры «Наборщик» выводится приветственное сообщение. Также для пользователя становятся доступны следующие опции: начать игру, ознакомиться с правилами игры, вывести турнирную таблицу или выйти из игры (рисунок 3).

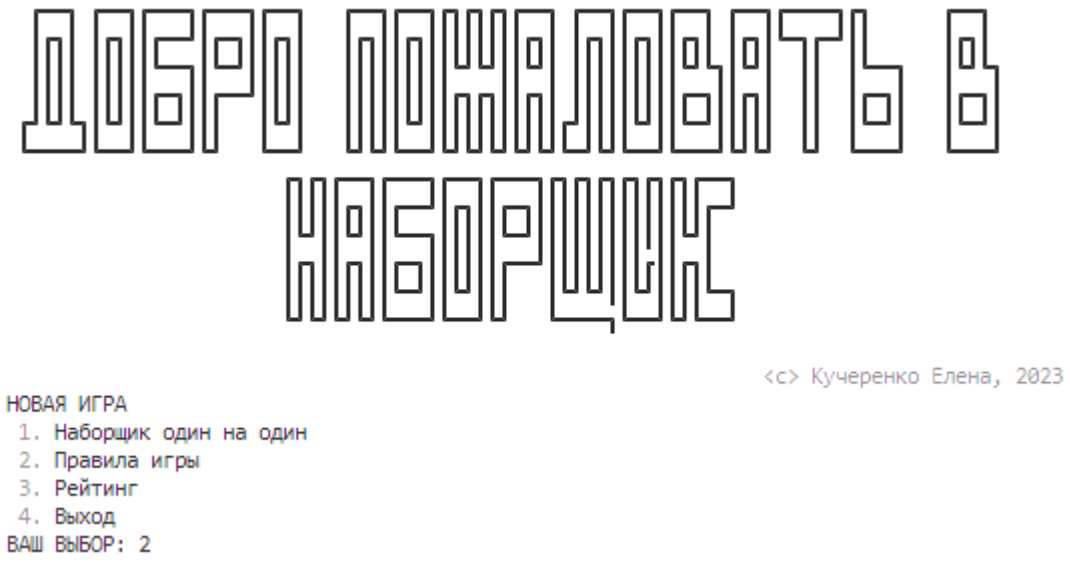


Рисунок 3 – Приветствие

Если введенный символ не будет соответствовать предоставленным (1, 2, 3, 4), то пользователю предложат выбрать еще раз (рисунок 4).

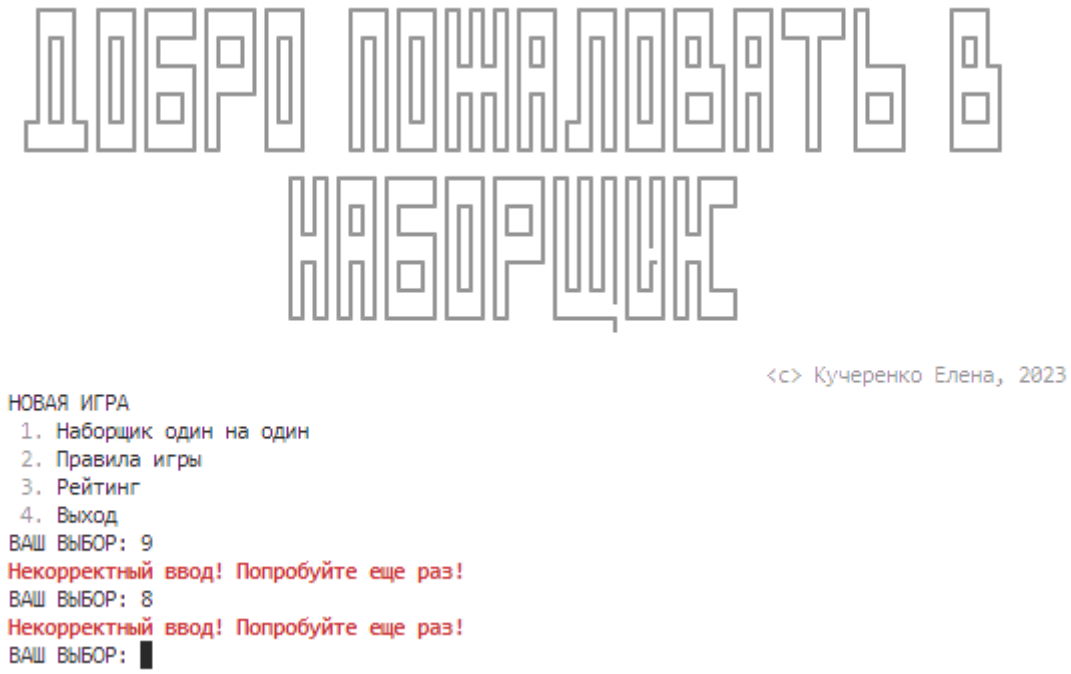


Рисунок 4 – Ошибка при вводе

При выборе игроком опции 2 выводятся правила игры, а также предлагается выбрать, что дальше хочет сделать игрок (рисунок 5).

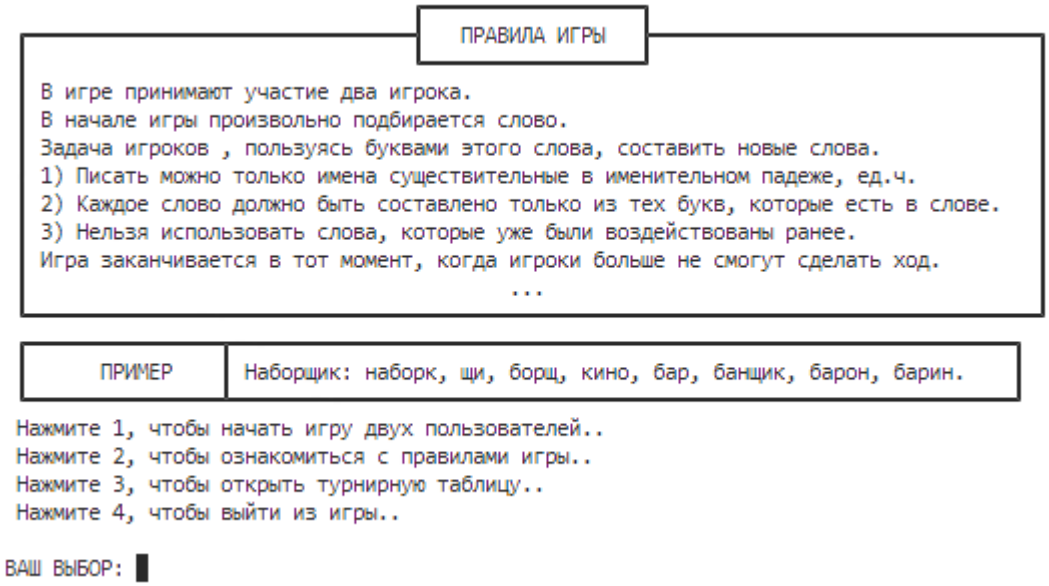


Рисунок 5 – Правила игры

При выборе игроком опции 3 выводятся правила игры, а также предлагается выбрать, что дальше хочет сделать игрок (рисунок 6). Если введенный символ не будет соответствовать предоставленным (1, 2, 3, 4), то пользователю предложат выбрать еще раз (рисунок 6).

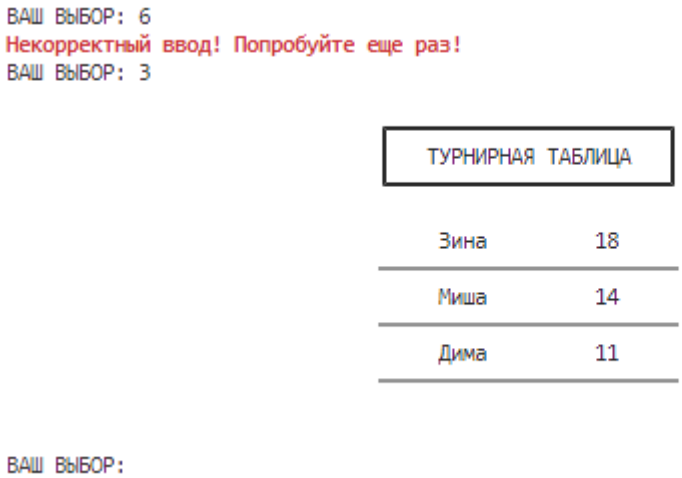


Рисунок 6 – Турнирная таблица

Вывод игрового слова. Из него составляются варианты ответов. Если совершил некорректный ввод, то игра сообщает об этом (рисунок 7).

Чтобы активировать слово–подсказку, пользователю необходимо набрать больше пятнадцати баллов и ввести слово «hint» (рисунок 8).

Если один из пользователей решил прервать игру, то он должен ввести слово «stop». Выведутся результаты матча и игра завершится (рисунок 9).

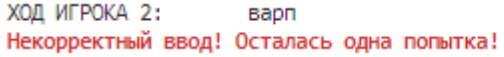


Рисунок 7 – Некорректный ввод

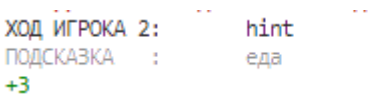


Рисунок 8 – Активирования слова–подсказки

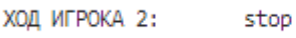


Рисунок 9 – Активирования слова–подсказки

Заключение

Во время выполнения курсовой работы были изучены принципы программирования на языке Си, были освоены основные концепции и стратегии по созданию консольного приложения «Наборщик», которое является результатом выполнения курсовой работы.

Библиографический список

1 Керниган Б. Язык программирования Си 2–е издание / Б. Керниган Д. Ричи М.: Издательский дом "Вильямс", 2012. 272 с.

2 HIT. Информационный сайт о высоких технологиях/ Режим доступа: http://all–ht.ru/about.html

3 Википедия. Свободная энциклопедия [Электронный ресурс] / Режим доступа: https://ru.wikipedia.org/wiki/Си_(язык_программирования)

Приложение А
(обязательное)
Код программы

```
#include <windows.h> //библиотека для работы с системой
#include <stdlib.h>  //содержит определения и прототипы функций общего пользования
#include <locale.h>  //отвечает за работу с локальной культурой
#include <string.h>  //для работы со строками
#include <ctype.h>   //отвечает за преобразование и проверку символов
#include <stdio.h>   //для работы со средствами ввода-вывода
#include <wchar.h>   //для работы с символами
#include <time.h>    //для работы с датами и временем

#define MAX_NUM_WORDS 68000
#define MAX_NUM_LETTERS 11
#define SIZE 20

#define DBOLD    "\033[2m"
#define GREEN    "\033[0;32m"
#define RESET    "\033[0m"
#define RED      "\033[1;31m"

char DICTIONARY_WORDS[MAX_NUM_WORDS][MAX_NUM_LETTERS];
char ENTERED_WORDS[MAX_NUM_WORDS][MAX_NUM_LETTERS];
char WORD_GENERATION[MAX_NUM_WORDS][MAX_NUM_LETTERS];



void Greeting(){
    printf("\n");
    printf("  ┌─┐ ┌─┐┌─┐┌─┐┌─┐┌─┐  ┌─┐┌─┐┌┐┌┌┐┌┌─┐ ┌─┐┌─┐┌─┐┌─┐┌─┐┌┐    ┌─┐\n");
    printf("  │┌┘ │┌┘│ ┌┘└─┐│┌┘└┐└┐ │┌┘└┐└┐└┐└┐└┐└┐ │┌┘└┐└┐└┐└┐└┐└┐
```

```
}

void Rules(){
    printf("                                                                                \n");
    printf("                                ╔═══════════════╗                               \n");
    printf(" ╔══════════════════════════════╣   ПРАВИЛА ИГРЫ  ╠══════════════════════════════╗\n");
    printf(" ║                              ╚═══════════════╝                               ║\n");
    printf(" ║ В игре принимают участие два игрока.                                         ║\n");
    printf(" ║ В начале игры произвольно подбирается слово.                                 ║\n");
    printf(" ║ Задача игроков , пользуясь буквами этого слова, составить новые слова.       ║\n");
    printf(" ║ 1) Писать можно только имена существительные в именительном падеже, ед.ч.    ║\n");
    printf(" ║ 2) Каждое слово должно быть составлено только из тех букв, которые есть в слове.  ║\n");
    printf(" ║ 3) Нельзя использовать слова, которые уже были воздействованы ранее.         ║\n");
    printf(" ║ Игра заканчивается в тот момент, когда игроки больше не смогут сделать ход.  ║\n");
    printf(" ║                                    ...                                       ║\n");
    printf(" ╚══════════════════════════════════════════════════════════════════════════════╝\n");
    printf(" ╔═══════════════╦══════════════════════════════════════════════════════════════╗\n");
    printf(" ║      ПРИМЕР   ║ Наборщик: наборк, щи, борщ, кино, бар, банщик, барон, барин.  ║\n");
    printf(" ╚═══════════════╩══════════════════════════════════════════════════════════════╝\n");
    printf(" Нажмите 1, чтобы начать игру двух пользователей.. \n");
    printf(" Нажмите 2, чтобы ознакомиться с правилами игры..  \n");
    printf(" Нажмите 3, чтобы открыть турнирную таблицу..      \n");
    printf(" Нажмите 4, чтобы выйти из игры..                  \n");
    printf("\n");
}
void Farewell(){
    printf("\n");
    printf("                     ╔═══════════════════════════════════════╗\n");
    printf("                     ║      СПАСИБО ЗА ИГРУ! ДО НОВЫХ ВСТРЕЧ!      ║\n");
    printf("                     ╚═══════════════════════════════════════╝\n");
}

struct Date{
    int gameCounter1;
    char name1[100];
    int gameCounter2;
    char name2[100];
    int score;
    char name[999];
};

int Compare(const void* a, const void* b) {
    struct Date* p1 = (struct Date*)a;
    struct Date* p2 = (struct Date*)b;
    return p2->score – p1->score;
}

int Parse(){
    FILE *file;
```

Листинг А.1, лист 2

```
    struct Date players[50];
    char line[100];
    int temp;
    char scoreString[50];
    int i, j, n = 0;
    char kolvo[50];
    file = fopen("proverka.txt", "a+");
    if (!file) {
        printf("%sНе удалось открыть файл!%s", RED, RESET);
        return 1;
    }
    while (fgets(line, sizeof line, file) != NULL){
        struct Date *player = &players[n++];
        i = 0;
        j = 0;
        while (line[i] != ':'){
            player->name[i] = line[i];
            i++;
        }
        player->name[i] = '\0';
        i++;
        while (line[i] != '\0'){
            scoreString[j] = line[i];
            i++;
            j++;
        }
        scoreString[j] = '\0';
        player->score = atoi(scoreString);
    }
    fclose(file);
    qsort(players, n, sizeof(struct Date), Compare);

    for(i = 0; i < n; i++){
        printf("                                    %s    \t %d\n", players[i].name, players[i].score);
        printf("%s                                 ======================%s\n", DBOLD, RESET);
    }
    return 0;
}

int FirstWritingToAFile(struct Date users){
    char line[100], arr2[100], arr1[100],  buffer[100], line2[100];
    int array2, a = 0, b = 0, c = 0, d = 0, m = 0;
    FILE *f = fopen("proverka.txt", "a+");
    while (fgets(line, sizeof line, f) != NULL){
        if (strstr(line, users.name1)){ //ЕСЛИ НАХОДИТ ИМЯ ИГРОКА, ТО...
            a++;
            int j = 0;
            int i = 0;
            while (line[i] != ':'){
```

Листинг А.1, лист 3

```
                arr1[i] = line[i];
                i++;
            }
            arr1[i] = '\0';
            i++;
            while (line[i] != '\0'){
                arr2[j] = line[i];
                i++;
                j++;
            }
            arr2[j] = '\0';
            array2 = atoi(arr2);
            if (users.gameCounter1 > array2){
                b++;
            }
        }
    }
    if (a == 0){
        fprintf(f, "%s:%i\n", users.name1, users.gameCounter1);
    }
    fclose(f);
    if (b != 0){
        FILE *l = fopen("proverka.txt", "r");
        FILE *file = fopen("itog.txt", "w+");
            while (fgets(buffer, sizeof buffer, l) != NULL){
                if (strstr(buffer, users.name1)){
                    fprintf(file, "%s:%i\n", users.name1, users.gameCounter1);
                }
                else {
                    fprintf(file, "%s", buffer);
                }
            }
        fclose(file);
        fclose(l);
        m++;
    }
    if (m != 0){
        FILE *li = fopen("proverka.txt", "w+");
        FILE *fi = fopen("itog.txt", "r");
            while (fgets(line2, sizeof line2, fi) != NULL){
                fprintf(li, "%s", line2);
            }
        fclose(fi);
        fclose(li);
    }
}

int SecondWritingToAFile(struct Date users){
    char line[100], arr2[100], arr1[100],  buffer[100], line2[100];
```

Листинг А.1, лист 4

```
int array2, a = 0, b = 0, c = 0, d = 0, m = 0;
FILE *f2 = fopen("proverka.txt", "a+");
while (fgets(line, sizeof line, f2) != NULL){
    if (strstr(line, users.name2)){ //ЕСЛИ НАХОДИТ ИМЯ ИГРОКА, ТО...
        a++;
        int j = 0;
        int i = 0;
        while (line[i] != ':'){
            arr1[i] = line[i];
            i++;
        }
        arr1[i] = '\0';
        i++;
        while (line[i] != '\0'){
            arr2[j] = line[i];
            i++;
            j++;
        }
        arr2[j] = '\0';
        array2 = atoi(arr2);
        if (users.gameCounter2 > array2){
            b++;
        }
    }
}
if (a == 0){
    fprintf(f2, "%s:%i\n", users.name2, users.gameCounter2);
}
fclose(f2);
if (b != 0){
    FILE *l2 = fopen("proverka.txt", "r");
    FILE *file2 = fopen("itog.txt", "w+");
        while (fgets(buffer, sizeof buffer, l2) != NULL){
            if (strstr(buffer, users.name2)){
                fprintf(file2, "%s:%i\n", users.name2, users.gameCounter2);
            }
            else {
                fprintf(file2, "%s", buffer);
            }
        }
    fclose(file2);
    fclose(l2);
    m++;
}
if (m != 0){
    FILE *li2 = fopen("proverka.txt", "w+");
    FILE *fi2 = fopen("itog.txt", "r");
        while (fgets(line2, sizeof line2, fi2) != NULL){
            fprintf(li2, "%s", line2);
```

Листинг А.1, лист 5

```
            }
        fclose(fi2);
        fclose(li2);
    }
}

int Check(char *s2){
    int q = 0, w = 0;
    char t[MAX_NUM_LETTERS];
    strcpy(t, s2);

    while (t[q] != '\0'){
        t[q] = tolower(t[q]);
        q++;
    }

    while (w < MAX_NUM_WORDS){
        if (!strcmp(t, DICTIONARY_WORDS[w])){
            return 1;
        }
        w++;
    }
    return 0;
}

int FirstCheck(char *s1, char *s2){
    int k = 0, n;
    char t[MAX_NUM_LETTERS];
    strcpy(t, s1);
    for (int i = 0; i < strlen(s2) && k != -1; i++){
        k = -1;
        for (int j = 0; j < strlen(s1) && k == -1; j++)
            if (toupper(s1[j]) == toupper(s2[i])){
                k = j;
            }
        if (k != -1){
            n = strlen(s1) - 1;
            for (int j = k; j < n; j++)
            s1[j] = s1[j+1];
            s1[n] = '\0';
        }
    }
    strcpy(s1, t);
    if (k != -1 && Check(s2)){
        return 1;
    }
    else {
        return 0;
    }
```

Листинг А.1, лист 6

```
}

int SecondCheck(char allWords[MAX_NUM_WORDS][MAX_NUM_LETTERS], int *number, char word[MAX_NUM_LETTERS]){
    int i = 0, k = -1;
    for (i; i < *number && k == -1; i++)
        if (stricmp(allWords[i], word) == 0) k = i;
    if (k == -1){
        strcpy(allWords[*number], word);
        (*number)++;
    }
    if (k != -1){
        return 1;
    }
    else {
        return 0;
    }
}

int ThePlayersMove(char *playWord, char ENTERED_WORDS[MAX_NUM_WORDS][MAX_NUM_LETTERS], int *counter, int *score, char
word[MAX_NUM_LETTERS]){
    int resultFirstCheck, resultSecondCheck, end = 0, resultFirstCheckh, resultSecondCheckh, n = 0, b = 0;
    if (!strcmp(word, "stop")){
        return 2;
    }
    if (!strcmp(word, "hint")){
        if (*score < 15){
            printf("%sУ вас недостаточно очков! %s", RED, RESET);
            return 0;
        }
        else {
            while (n == 0){
                srand(time(NULL)); // инициализация генератора псевдослучайных чисел
                // int randomIndex = 0 + rand()%(MAX_NUM_WORDS - 1 + 1); // генерация случайного индекс
                memcpy(word, DICTIONARY_WORDS[rand() % MAX_NUM_WORDS - b], 11);
                resultFirstCheck = FirstCheck(playWord, word);
                if (resultFirstCheck == 1){
                    resultSecondCheck = SecondCheck(ENTERED_WORDS, counter, word);
                    if (resultSecondCheck == 0){
                        n++;
                    }
                }
                b++;
            }
            printf("%s", DBOLD);
            printf("ПОДСКАЗКА   :       %s\n", word);
            printf("%s", RESET);

            return 1;
        }
```

Листинг А.1, лист 7

```
    }
    do{
        resultFirstCheck = FirstCheck(playWord, word);
        if (resultFirstCheck == 1){
            resultSecondCheck = SecondCheck(ENTERED_WORDS, counter, word);
            if (resultSecondCheck == 1){
                printf("%sТакое слово уже использовалось! %s", RED, RESET);
                end ++;
                resultSecondCheck --;
            }
        }
    }
    while (resultFirstCheck == 1 && resultSecondCheck == 1);
    if (end != 0){
        resultFirstCheck = 0;
        return resultFirstCheck;
    }
    else if (resultFirstCheck != 0){
        *score = *score + strlen(word);
        return resultFirstCheck;
    }
}

int FillDictionary(int *number){
    FILE *f = fopen("dictionary.txt", "r");
    if (!f){
        return 0;
    }
    else {
        *number = 0;
        while (*number < MAX_NUM_WORDS && fscanf(f, "%s", DICTIONARY_WORDS[(*number)++]) > 0);
        (*number)--;
        fclose(f);
        if (*number == 0){
            return 0;
        }
        else {
            return 1;
        }
    }
}

int Score(int player1, int player2, struct Date users){
    users.gameCounter1  = player1;
    users.gameCounter2  = player2;
    FirstWritingToAFile(users);
    SecondWritingToAFile(users);
    if (player1 > player2){
        printf("\n");
```

Листинг А.1, лист 8

```
        printf("                            ПОБЕДУ ОДЕРЖАЛ ПЕРВЫЙ ИГРОК! СЧЕТ:\n");
        printf("                        ╔═══════════════════╦═══════════════════╗\n");
        if (player1 < 10){
            printf("                        ║         %i        ║", player1);
        }
        else printf("                        ║         %i        ║", player1);
        if (player2 < 10){
            printf("         %i         ║\n", player2);
        }
        else printf("          %i        ║\n", player2);
        printf("                        ╚═══════════════════╩═══════════════════╝\n");
        printf("\n");
        Farewell();
    }
    else if (player1 < player2){
        printf("\n");
        printf("                            ПОБЕДУ ОДЕРЖАЛ ВТОРОЙ ИГРОК! СЧЕТ:\n");
        printf("                        ╔═══════════════════╦═══════════════════╗\n");
        if (player1 < 10){
            printf("                        ║         %i        ║", player1);
        }
        else printf("                        ║         %i        ║", player1);
        if (player2 < 10){
            printf("         %i         ║\n", player2);
        }
        else printf("          %i        ║\n", player2);
        printf("                        ╚═══════════════════╩═══════════════════╝\n");
        printf("\n");
        Farewell();
    }
    else if (player1 = player2){
        printf("\n");
        printf("\n                                        НИЧЬЯ! СЧЕТ:\n");
        printf("                        ╔═══════════════════╦═══════════════════╗\n");
        if (player1 < 10){
            printf("                        ║         %i        ║", player1);
        }
        else printf("                        ║         %i        ║", player1);
        if (player2 < 10){
            printf("         %i         ║\n", player2);
        }
        else printf("          %i        ║\n", player2);
        printf("                        ╚═══════════════════╩═══════════════════╝\n");
        printf("\n");
        Farewell();
        }
}

void Rating(){
```

Листинг А.1, лист 9

```
    printf("\n");
    printf("                                ╔═══════════════════╗\n");
    printf("                                ║   ТУРНИРНАЯ ТАБЛИЦА   ║\n");
    printf("                                ╚═══════════════════╝\n");
    printf("\n");
    Parse();
    printf("\n");
    printf("\n");
}

int main(){
    system("cls");
    char playWord[SIZE];
    char word[MAX_NUM_LETTERS];
    int result, score1 = 0, score2 = 0, counter = 0, countWords = 0, stopWhile = 0, n = 0;
    struct Date users;
    Greeting();
    int choice;
    printf("ВАШ ВЫБОР: ");
    scanf("%i", &choice);
    while (choice != 4){
        while (choice != 1 && choice != 2 && choice != 3 && choice != 4){
            printf("%sНекорректный ввод! Попробуйте еще раз!\n%s", RED,    RESET);
            printf("ВАШ ВЫБОР: ");
            scanf("%i", &choice);
        }
        if (choice == 1){
            while (stopWhile != 1){
                FILE *file = fopen("dictionary.txt", "r");
                char *fileContents[MAX_NUM_WORDS+1];
                char buffer[MAX_NUM_WORDS + 1];
                for (size_t i = 0; fgets(buffer, sizeof buffer, file); i++){
                    if (strlen(buffer) == 11){
                        n++;
                        fileContents[n] = strdup(buffer);
                    }
                }
                fclose(file);
                srand(time(NULL)); // инициализация генератора псевдослучайных чисел
                int randomIndex = 0 + rand()%(n - 1 + 1); // генерация случайного индекса
                if (randomIndex == 0){
                    randomIndex++;
                }
                memcpy(playWord, fileContents[randomIndex], 11);
                result = FillDictionary(&countWords);
                if (result == 0){
                    printf("%s Не удалось загрузить словарь! Попробуйте зайти в игру еще раз!\n%s ", RED,    RESET);
                    return 0;
```

Листинг А.1, лист 10

```
    }
    printf("\n%sИМЯ ИГРОКА 1:       ", DBOLD);
    scanf("%s", &users.name1);
    printf("ИМЯ ИГРОКА 2:       ");
    scanf("%s", &users.name2);
    printf("%s", RESET);
    printf("\n");
    printf("                              ИГРА НАЧАЛАСЬ! ЗАГАДАННОЕ СЛОВО:\n");
    printf("                                  ╔═══════════════════╗\n");
    printf("                                        %s", playWord);
    printf("                                  ╚═══════════════════╝\n");
    printf("\n");
    stopWhile++;
}
printf("ХОД ИГРОКА 1:       ");
scanf("%s", &word);
result = ThePlayersMove(playWord, ENTERED_WORDS, &counter,  &score1, word);
if (result == 0){
    printf("%sНекорректный ввод! Осталась одна попытка!\n%s", RED,    RESET);
    printf("ХОД ИГРОКА 1:       ");
    scanf("%s", &word);
    result = ThePlayersMove(playWord, ENTERED_WORDS, &counter,  &score1, word);
    if (result == 0){
        Score(score1, score2, users);
        break;
    }
}
else if (result == 2){
    Score(score1, score2, users);
    break;
}
printf("%s+%d\n%s", GREEN, strlen(word), RESET);

printf("ХОД ИГРОКА 2:       ");
scanf("%s", &word);
result = ThePlayersMove(playWord, ENTERED_WORDS, &counter, &score2, word);
if (result == 0){
    printf("%sНекорректный ввод! Осталась одна попытка!\n%s", RED,    RESET);
    printf("ХОД ИГРОКА 2:       ");
    scanf("%s", &word);
    result = ThePlayersMove(playWord, ENTERED_WORDS, &counter,  &score2, word);
    if (result == 0){
        Score(score1, score2, users);
        break;
    }
}
else if (result == 2){
    Score(score1, score2, users);
    break;
```

Листинг А.1, лист 11

```
            }
            printf("%s+%d\n%s", GREEN, strlen(word), RESET);
        }
        else if (choice == 2){
            Rules();
            printf ("ВАШ ВЫБОР: ");
            scanf("%i", &choice);
        }
        else if (choice == 3){
            Rating();
            printf ("ВАШ ВЫБОР: ");
            scanf("%i", &choice);
        }
    }
    if (choice == 4){
        Farewell();
    }
}
```

Листинг А.1, лист 12

Приложение Б
(обязательное)
Пример работы программы

Игра начинается с приветствия, после которого предлагается выбрать одну из трех опций (рисунок Б.1).

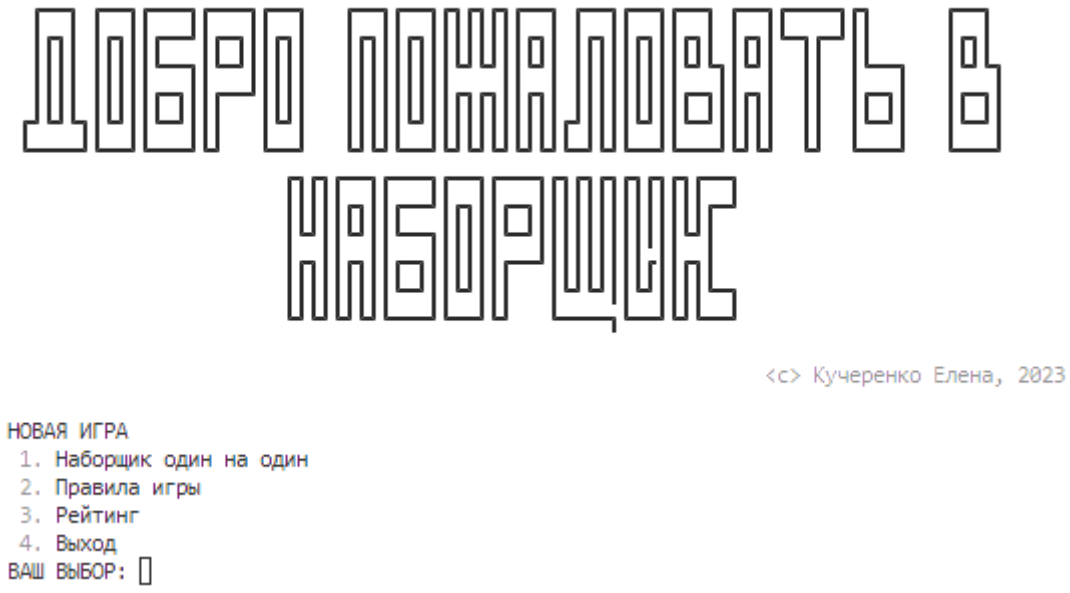


Рисунок Б.1 – Приветствие

Ознакомление с правилами (рисунок Б.2).

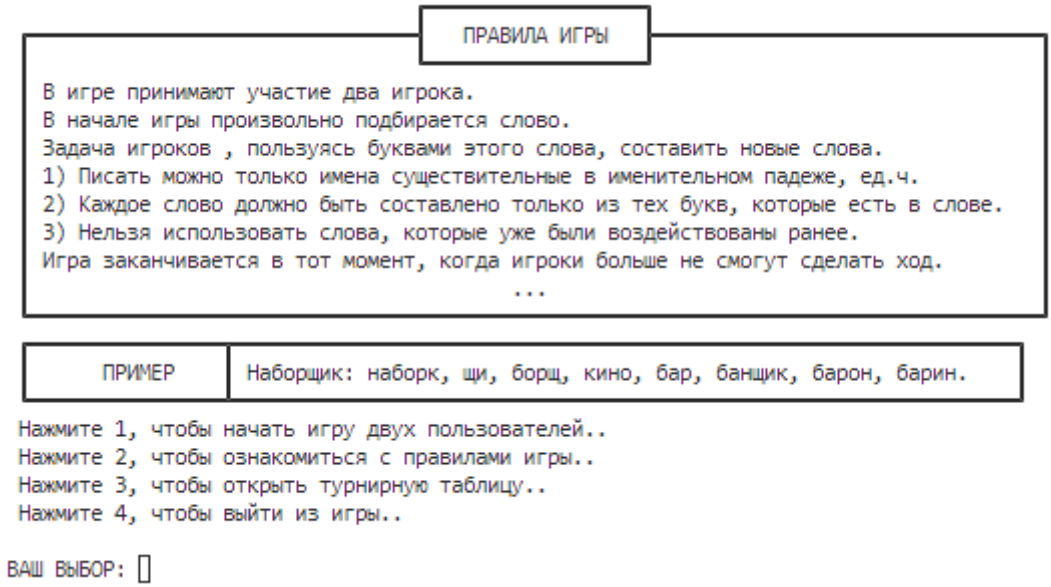


Рисунок Б.2 – Правила игры

Вывод турнирной таблицы (рисунок Б.3).

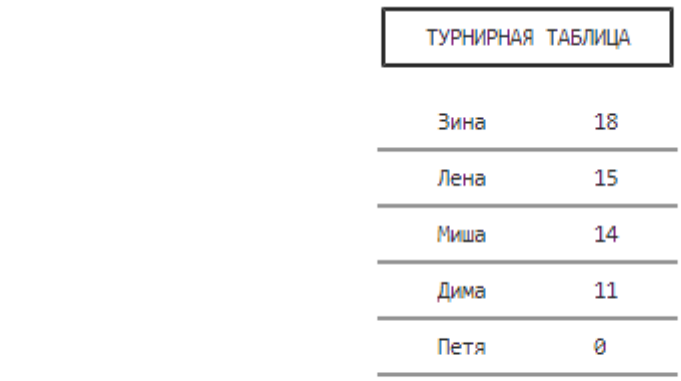


Рисунок Б.3 – Турнирная таблица

Начало игры. Ввод имен игроков (рисунок Б.4).

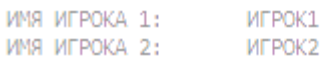


Рисунок Б.4 – Ввод имен

Генерация и вывод игрового слова (рисунок Б.5).

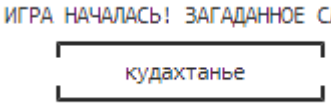


Рисунок Б.5 – Вывод игрового слова

Игрок номер один делает ход, пишет неподходящее слово. Игра его не засчитывает и выводит ошибку (рисунок Б.6). Первый игрок делает повторный ввод, слово подходит, игрок получает заработанные баллы (рисунок Б.7).

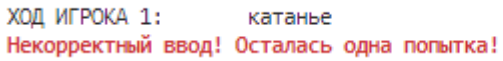


Рисунок Б.6 – Некорректный ввод

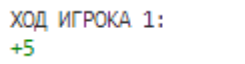


Рисунок Б.7 – Ход первого игрока

Игрок номер два сделал успешный ход (рисунок Б.8).

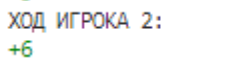


Рисунок Б.8 – Ход второго игрока

Игрок номер один делает ход, пишет слово, которое уже использовалось в игре. Игра его не засчитывает и выводит ошибку (рисунок Б.9). Первый игрок делает повторный ввод, слово подходит, игрок получает заработанные баллы (рисунок Б.10).



Рисунок Б.9 – Ход второго игрока

Игроки делают ряд успешных ходов (рисунок Б.10).



Рисунок Б.10 – Ход второго игрока

Игрок номер два делает неудачный ход. Игра его не засчитывает и выводит ошибку (рисунок Б.11). Второй игрок пользуется подсказкой (рисунок Б.12).

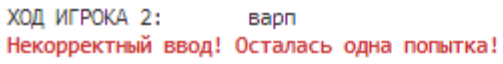


Рисунок Б.11 – Ход второго игрока

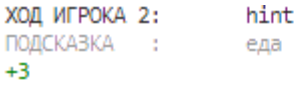


Рисунок Б.12 – Ход второго игрока

Первый игрок сделал успешный ход (рисунок Б.13).

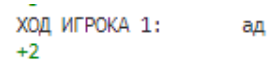


Рисунок Б.13 – Ход второго игрока

Второй игрок решает завершить игру (рисунок Б.14).

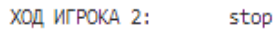


Рисунок Б.14 – Ход второго игрока

Выводятся результаты игры (рисунок Б.15).

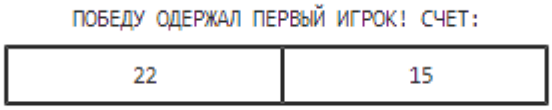


Рисунок Б.15 – Ход второго игрока

Прощание (рисунок Б.16).

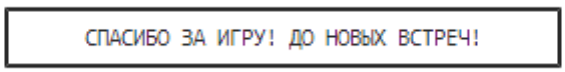


Рисунок Б.16 – Ход второго игрока